Universidade do Minho

Ano Letivo: 2026/27

Sistemas Conexionistas para Aprendizagem Automática

PL1 – Tooling Overview



Licenciatura em Ciência de Dados, 1º Semestre, 3º Ano

Sumário

1 Anaconda

2 PyTorch

3 Soluções na Cloud

4 Transformação de dados

TOOLING









TOOLING

Anaconda

O que é?

- A plataforma de distribuição Python *open-source* mais popular do mundo.
- Distribuição Python focada em ciência de dados e computação científica.
- Altamente suportado pela comunidade.
- Inclui gestor de pacotes/envs (**conda**), Jupyter Notebooks e muitas bibliotecas.



ANACONDA

TOOLING

Anaconda

Quais são as vantagens? **Isolamento de ambientes → evita conflitos de bibliotecas**

- O Anaconda permite-nos criar diferentes ambientes de trabalho dentro do nosso computador.
- Podemos pensar num ambiente de trabalho como uma divisão de uma casa.





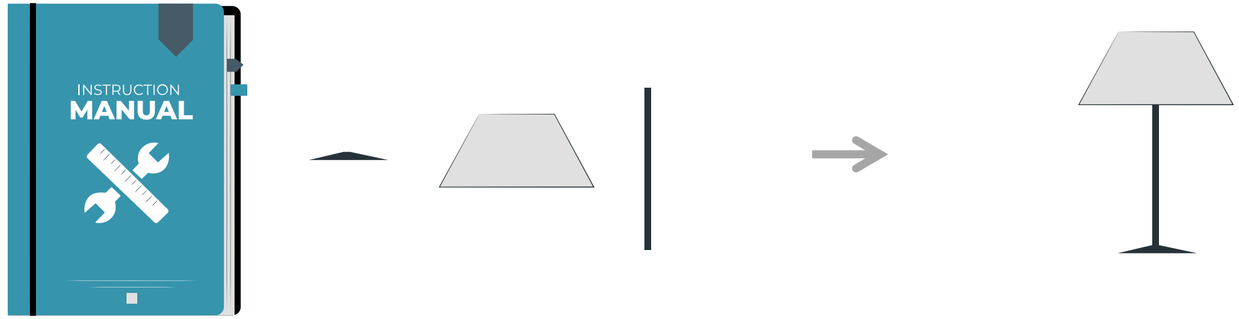
ANACONDA

TOOLING

Anaconda

Quais são as vantagens? **Instalação simplificada de *packages***

É como comprar um móvel já montado em vez de receber as peças e ter de seguir um manual de 50 passos.



✗ Instalar pacotes como NumPy ou SciPy diretamente via *pip* pode falhar ou demorar, pois é necessário compilar código-fonte e bibliotecas nativas (C/C++/Fortran).

✓ **Com o Anaconda:**

- Pacotes já vêm pré-compilados e otimizados para o sistema operativo.
- Um comando simples (*conda install numpy*) instala tudo, sem precisar de configurar compiladores.

ANACONDA

TOOLING

Anaconda

Quais são as limitações?

- Imagem inicial grande (ocupação em disco).
- Atualizações e gestão de versões podem exigir atenção (conda vs pip).
- Menos conveniente para colaboração em tempo real (comparado com Colab/Kaggle).

Quando deve ser utilizado?

- Desenvolver localmente com dependências específicas.
- Reproduzir ambientes entre equipas e máquinas.
- Projetos *offline* ou com restrições de segurança que impedem usar *notebooks* na *cloud*.

ANACONDA

TOOLING

Anaconda

Anaconda ≠ Conda

Anaconda

O que é? é o uma distribuição de *software* para ciência de dados que inclui um conjunto útil de *packages*, incluindo o próprio conda.



É como comprar um *kit* completo de ferramentas já com a caixa, as chaves de fendas, martelo e fita métrica incluídos.

Conda

O que é? é a ferramenta que gere *packages*, dependências e ambientes de trabalho e permite:

- criar, guardar, carregar e alterar de forma simples os ambientes de trabalho;
- instalar, atualizar e executar facilmente qualquer *package* (e as suas dependências… automaticamente!)



É como ter **apenas** a caixa de ferramentas — podes enchê-la com o que quiseres, mas começa vazia.

ANACONDA®

TOOLING

Anaconda

Componentes Principais:

- **Conda** — gerir ambientes e dependências (criar, ativar, instalar).
- **Jupyter / JupyterLab** — notebooks interativos para exploração e apresentação.
- **Pacotes incluídos** — NumPy, pandas, Matplotlib, scikit-learn, etc.

O Anaconda oferece duas interfaces principais para interação:

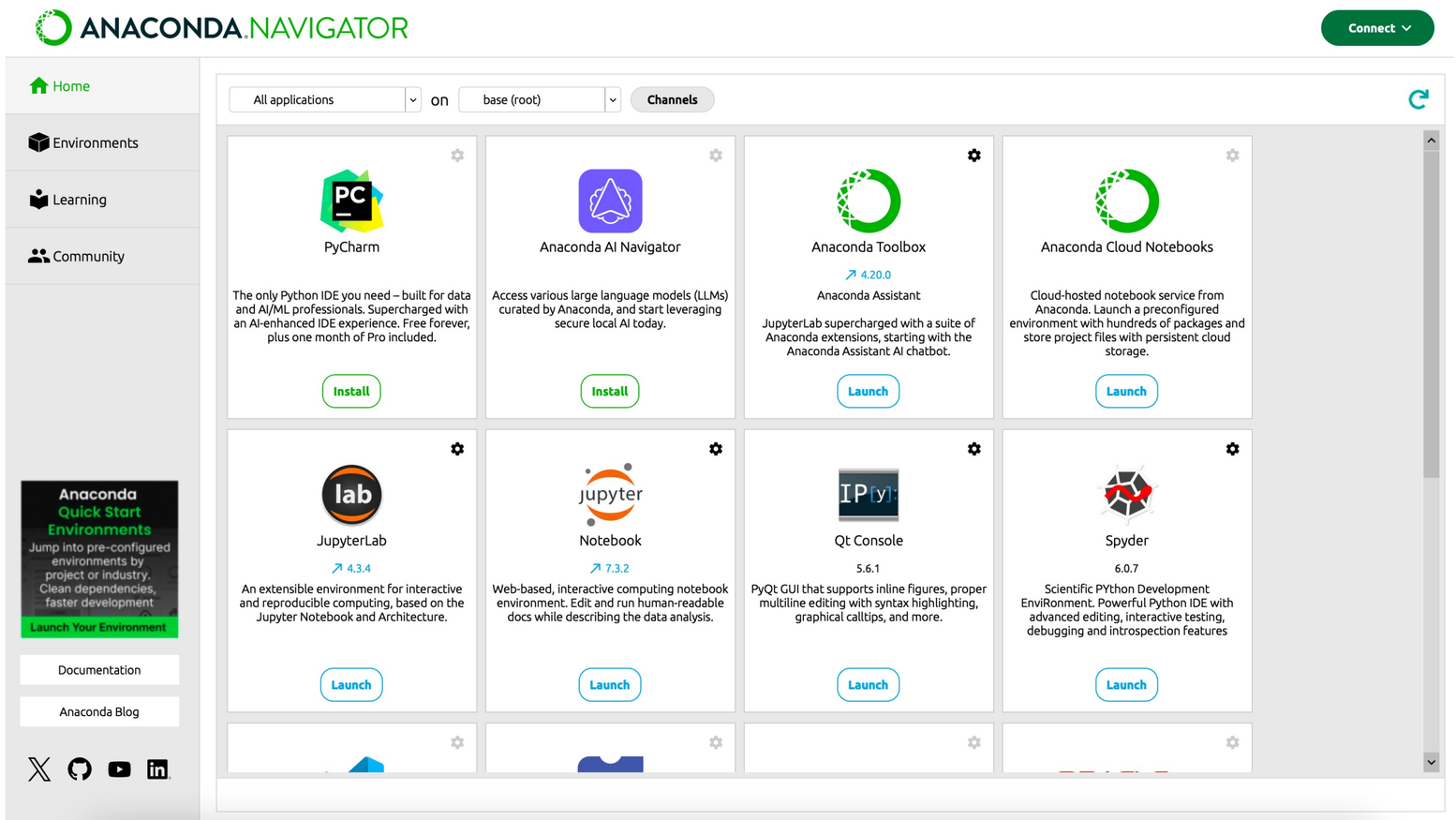
- **Anaconda Navigator** — É uma interface gráfica para gerir ambientes e iniciar aplicações, permitindo instalar, executar e atualizar aplicações como Jupyter Notebook e Spyder sem usar o terminal.
- **Anaconda Prompt (ou o terminal para Linux e macOS)** — É uma interface de linha de comandos que permite interagir com o sistema usando comandos do Conda.

ANACONDA

TOOLING

Anaconda

Anaconda Navigator



ANACONDA

TOOLING

Anaconda

Anaconda Prompt (or Terminal on Linux and macOS)



ANACONDA

TOOLING

Anaconda

IDEs para todos os gostos:

- VS Code
- Spyder (Scientific PYthon Development EnviRonment)
- Jupyter Notebooks
- PyCharm
- PyDev

  Python IDE for Eclipse
- ...

Alternativas:

- Pip + Virtualenv
- + brew
- Miniconda
- ...

ANACONDA

TOOLING

Anaconda

Instalação Windows/MacOS

1. Aceder a *https://www.anaconda.com/download*
2. Fazer download do ficheiro
3. Executar
4. Clique em Avançar e em Concordo para aceitar os Termos de Serviço (TOS) da Anaconda.
5. Iniciar

Instalação Linux x86:

- Seguir o tutorial:

  https://www.anaconda.com/docs/getting-started/anaconda/install#macos-linux-installation

ANACONDA

TOOLING

Anaconda

Aplicação Prática:

- Experimentar Conda e/ou Anaconda Navigator
- Criar um novo ambiente para as aulas usando python v3.12

  ex.: `conda create --name lcd_scaa python=3.12`
  `conda create -n lcd_scaa python=3.12`

- Instalar novos packages como o pytorch, pandas, numpy, matplotlib e scikit-learn

  ex.:
  ```
  conda activate lcd_scaa
  conda install -c conda-forge pytorch
  conda install numpy
  ...
  ```

- Verificar os packages instalados

  ex.: `conda list`

- Desativar o ambiente

  ex.: `conda deactivate`

⚠ *https://anaconda.org/anaconda/repo*

ANACONDA

TOOLING

Anaconda

Aplicação Prática:

- Criar um novo ambiente com python v3.9

  ex.: `conda create -n env39 python=3.9`

- Ativar o ambiente

  ex.: `conda activate env39`

- Verificar os ambientes disponíveis

  ex.: `conda env list`

- Experimentar o Jupyter

  ex.: `jupyter notebook`

  ```
  print("Hello World")
  ```

- Instalar e experimentar VSCode

```
print("Hello World")
```

ANACONDA

TOOLING

Anaconda

Anaconda Prompt (or Terminal on Linux and macOS)

Conda basics	
Verify conda is installed, check version number	`conda info`
Update conda to the current version	`conda update conda`
Install a package included in Anaconda	`conda install PACKAGENAME`
Run a package after install, example Spyder*	`spyder`
Update any installed program	`conda update PACKAGENAME`
Command line help	`COMMANDNAME --help` `conda install --help`
Using environments	
Create a new environment named py35, install Python 3.5	`conda create --name py35 python=3.5`
Activate the new environment to use it	`WINDOWS: activate py35` `LINUX, macOS: source activate py35`
Get a list of all my environments, active environment is shown with *	`conda env list`
Make exact copy of an environment	`conda create --clone py35 --name py35-2`
List all packages and versions installed in active environment	`conda list`
List the history of each change to the current environment	`conda list --revisions`
Restore environment to a previous revision	`conda install --revision 2`
Save environment to a text file	`conda list --explicit > bio-env.txt`
Delete an environment and everything in it	`conda env remove --name bio-env`
Deactivate the current environment	`WINDOWS: deactivate` `macOS, LINUX: source deactivate`
Create environment from a text file	`conda env create --file bio-env.txt`
Stack commands: create a new environment, name it bio-env and install the biopython package	`conda create --name bio-env biopython`

https://docs.conda.io/projects/conda/en/4.6.0/_downloads/52a95608c49671267e40c689e0bc00ca/conda-cheatsheet.pdf

ANACONDA

TOOLING

Anaconda

Anaconda Prompt (or Terminal on Linux and macOS)

Installing and updating packages	
Install a new package (Jupyter Notebook) in the active environment	`conda install jupyter`
Run an installed package (Jupyter Notebook)	`jupyter-notebook`
Install a new package (toolz) in a different environment (bio-env)	`conda install --name bio-env toolz`
Update a package in the current environment	`conda update scikit-learn`
Install a package (boltons) from a specific channel (conda-forge)	`conda install --channel conda-forge boltons`
Install a package directly from PyPI into the current active environment using pip	`pip install boltons`
Remove one or more packages (toolz, boltons) from a specific environment (bio-env)	`conda remove --name bio-env toolz boltons`
Managing multiple versions of Python	
Install different version of Python in a new environment named py34	`conda create --name py34 python=3.4`
Switch to the new environment that has a different version of Python	`Windows: activate py34` `Linux, macOS: source activate py34`
Show the locations of all versions of Python that are currently in the path **NOTE**: The first version of Python in the list will be executed.	`Windows: where python` `Linux, macOS: which -a python`
Show version information for the current active Python	`python --version`

https://docs.conda.io/projects/conda/en/4.6.0/_downloads/52a95608c49671267e40c689e0bc00ca/conda-cheatsheet.pdf

ANACONDA

TOOLING

PyTorch

O que é?

Uma biblioteca open-source para computação numérica e deep learning, criada pela Meta AI e hoje mantida pela PyTorch Foundation.

Tensores

Estruturas como arrays do NumPy, mas com suporte nativo a GPU.

Autograd

Diferenciação automática: calcula gradientes para treinar redes neuronais.

Ecossistema

Usado em investigação e indústria: Meta, Tesla, OpenAI, Hugging Face.

PyTorch

TOOLING

PyTorch

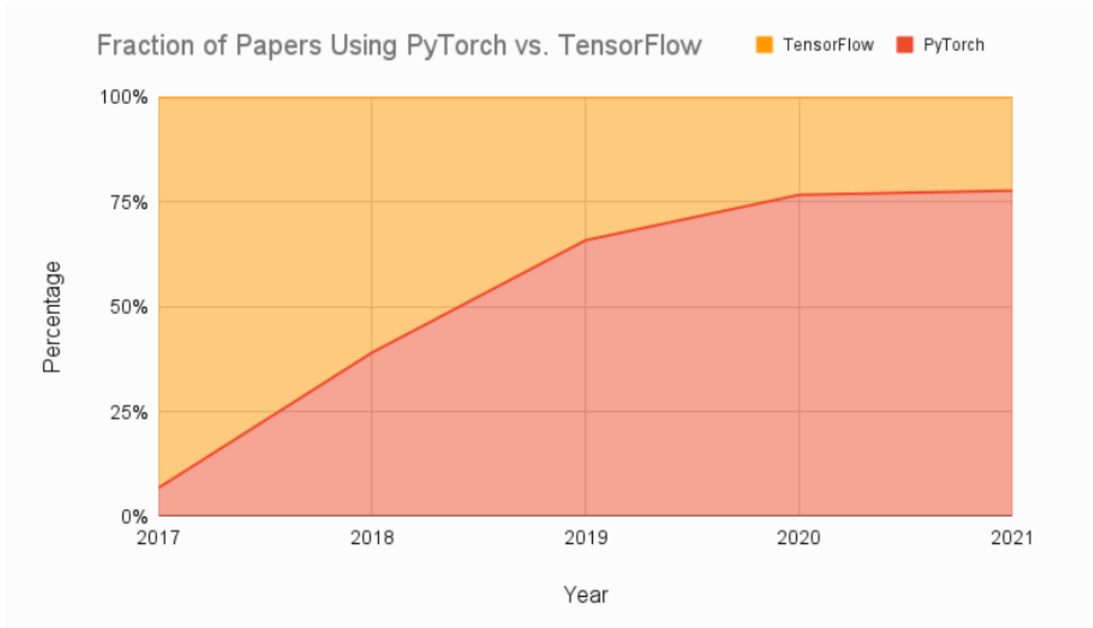
Quais são as vantagens?

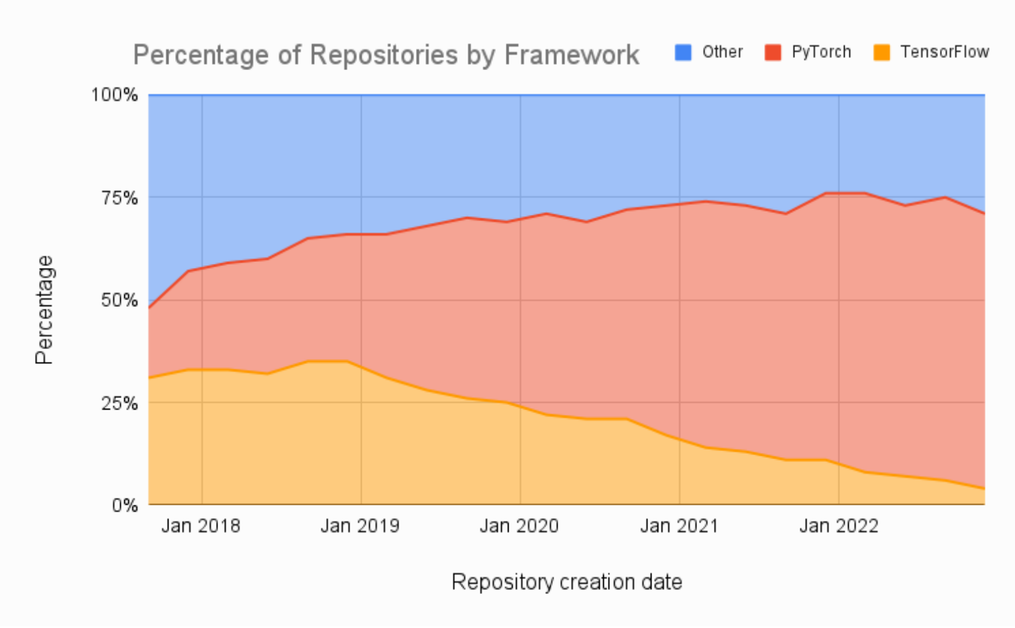
- **Grafo dinâmico (define-by-run)** — a rede é construída à medida que o código corre, o que facilita depuração e experimentação
- **API Pythonic** — integra-se naturalmente com NumPy e o resto do ecossistema Python
- **Suporte nativo a GPU** — mudar de CPU para GPU exige poucas alterações de código
- **Autograd** — diferenciação automática integrada, sem derivar gradientes à mão
- **Padrão em investigação** — a maioria dos artigos científicos recentes em deep learning publica código em PyTorch
- **Ecossistema maduro** — Hugging Face, torchvision, PyTorch Lightning, entre outros

PyTorch

TOOLING

PyTorch

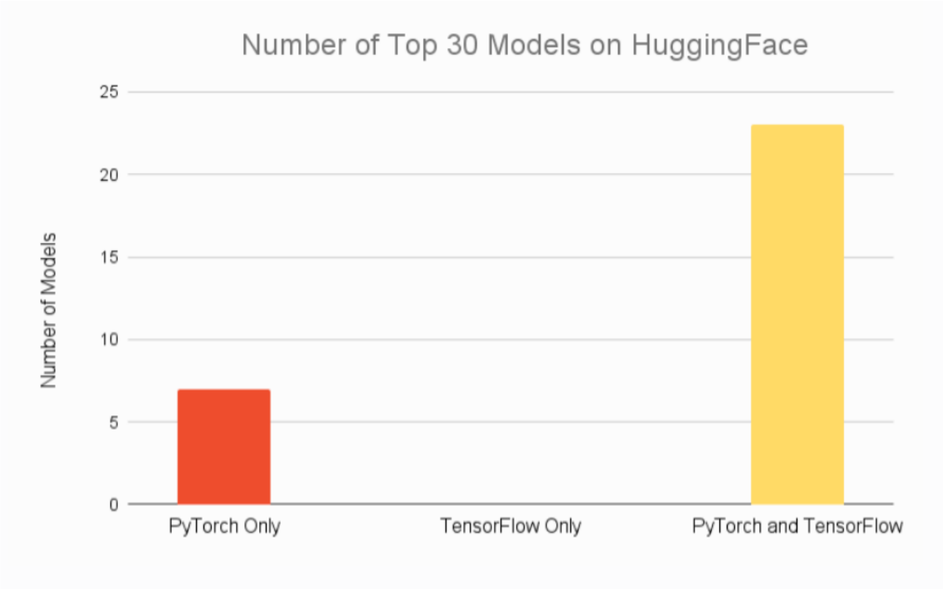


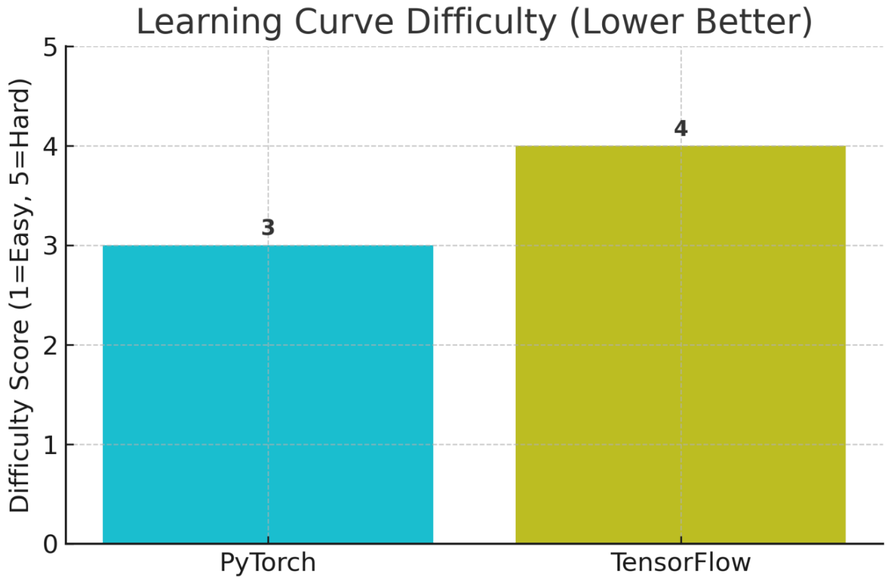


PyTorch

TOOLING

PyTorch





PyTorch

TOOLING

PyTorch

Alternativas:

TensorFlow

theano

Caffe2

Keras

PyTorch

TOOLING

PyTorch

Quais são as limitações?

- Deployment mobile/edge menos maduro que o TensorFlow.
- Deployment web mais fraco.
- Métricas não vêm por *default*.
- Menor adoção em ambientes corporativos legados

Quando deve ser utilizado?

- Investigação e prototipagem, sendo o mais utilizado em papers recentes.
- Ensino que queira mostrar o mecanismo interno do treino.
- NLP e LLMs, dada a base do Hugging Face Transformers.
- Reinforcement Learning, dada a base do Stable-Baselines3 e Gymnasium.
- Projetos onde a comunidade e colaboradores já usam PyTorch.

PyTorch

TOOLING

PyTorch

Empresas

Meta

OpenAI

Microsoft

aws

LinkedIn

intel

PyTorch

TOOLING

Python Libraries

NumPy

- Base para computação científica em Python.
- Suporte a **arrays multidimensionais** e a operações matemáticas de alto desempenho.
- Muito mais eficiente que listas Python nativas.





pandas

- Biblioteca para **análise e manipulação de dados**.
- Estruturas principais: **DataFrame** (tabelas) e **Series** (colunas).
- Ferramentas para limpeza, transformação e exploração de dados.

TOOLING

Python Libraries

Matplotlib

- Biblioteca para produzir **visualizações de dados 2D**.
- Criação de gráficos estáticos (linhas, barras, dispersão, histogramas, etc.).
- Base para outras bibliotecas como **Seaborn**.





scikit-learn

- Biblioteca para **Machine Learning tradicional**.
- Inclui algoritmos de classificação, regressão, *clustering* e pré-processamento.
- Ferramentas simples e eficientes para mineração e análise de dados.

TOOLING

Soluções na cloud



colab

kaggle

amazon web services™

Google Cloud

Microsoft Azure

GPU EATER

TOOLING

Colab

O que é?

- Ambiente Jupyter Notebook baseado na *cloud* criado pela Google.
- Funciona inteiramente no navegador (sem necessidade de configuração).
- Suporta Python com bibliotecas de ciência de dados e ML pré-instaladas (TensorFlow, PyTorch, scikit-learn, etc.).
- Acesso gratuito a GPU e TPU (tempo limitado por sessão).
- Integração com o github e o Google Drive → armazena e carregua notebooks facilmente.
- Fácil partilha e colaboração, tal como no Google Docs.
- Suporta Markdown, código e visualizações num único local.

colab

TOOLING

Colab

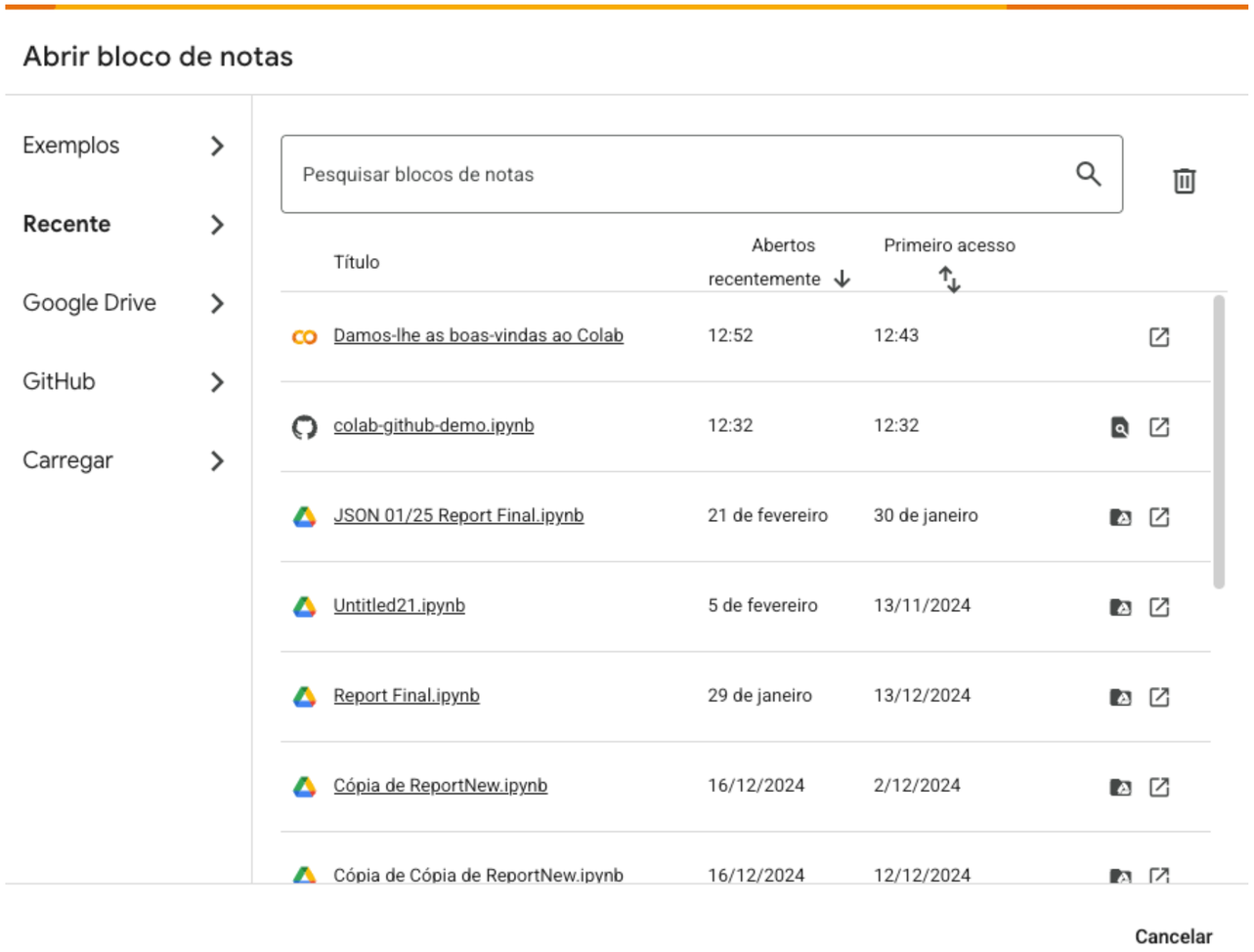
https://colab.research.google.com/



TOOLING

Colab

Abrir um notebook

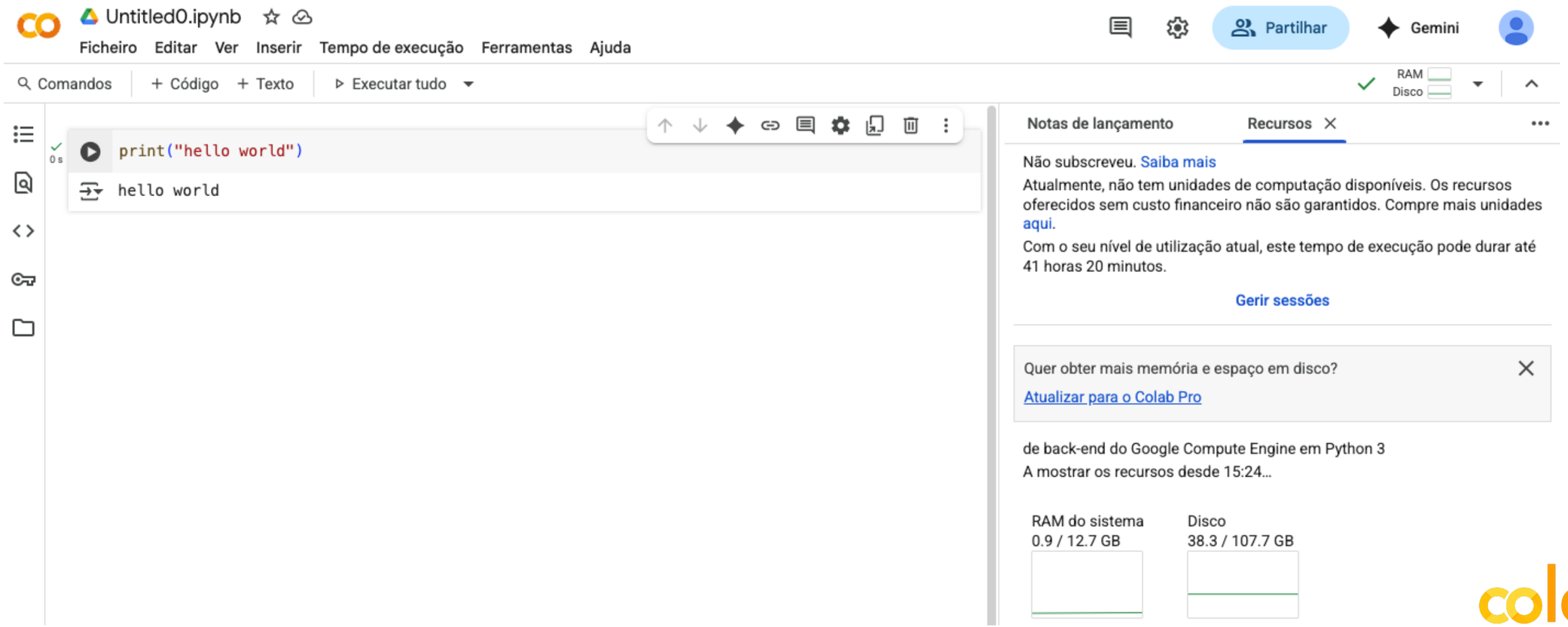


colab

TOOLING

Colab

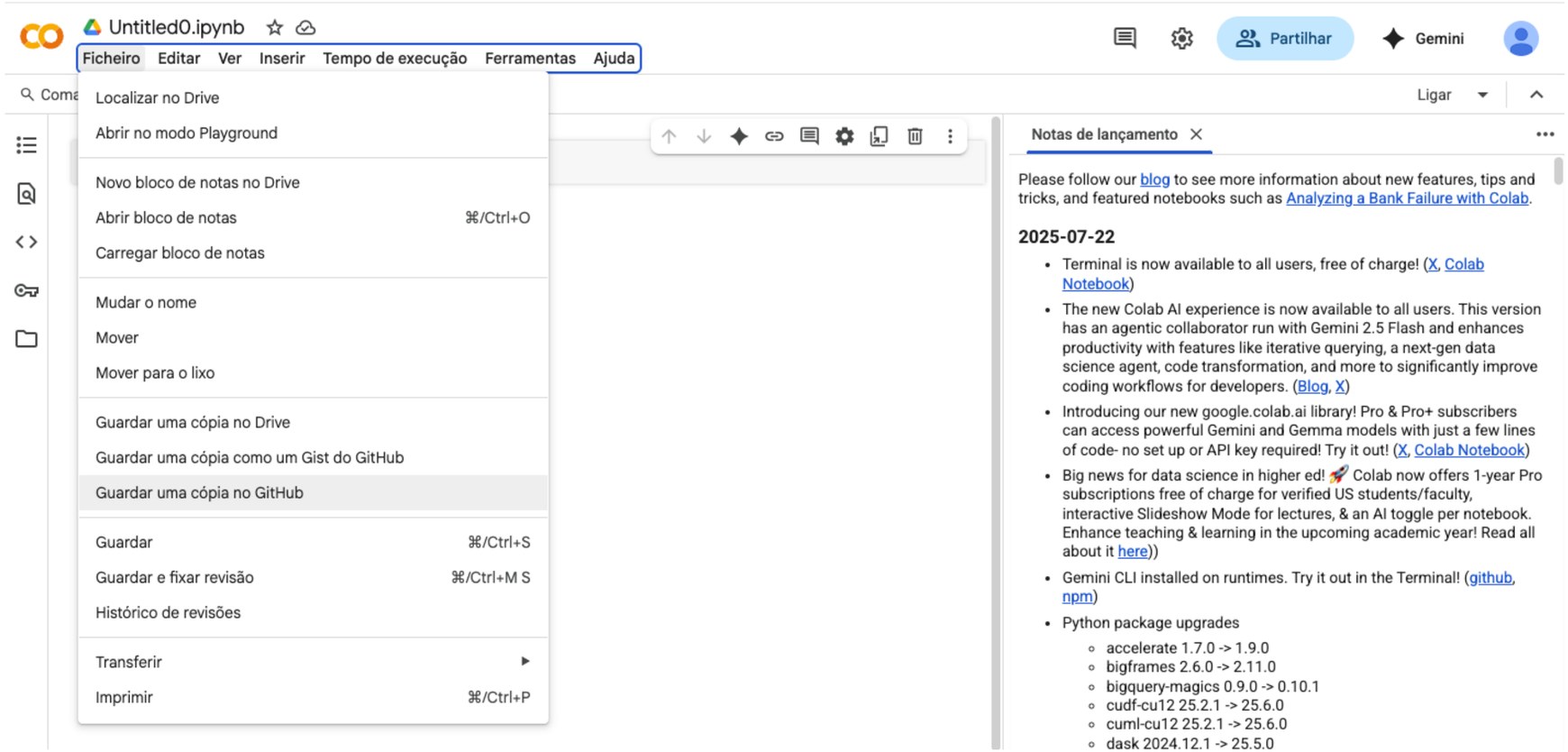
Criar um notebook



TOOLING

Colab

Guardar uma cópia no GitHub

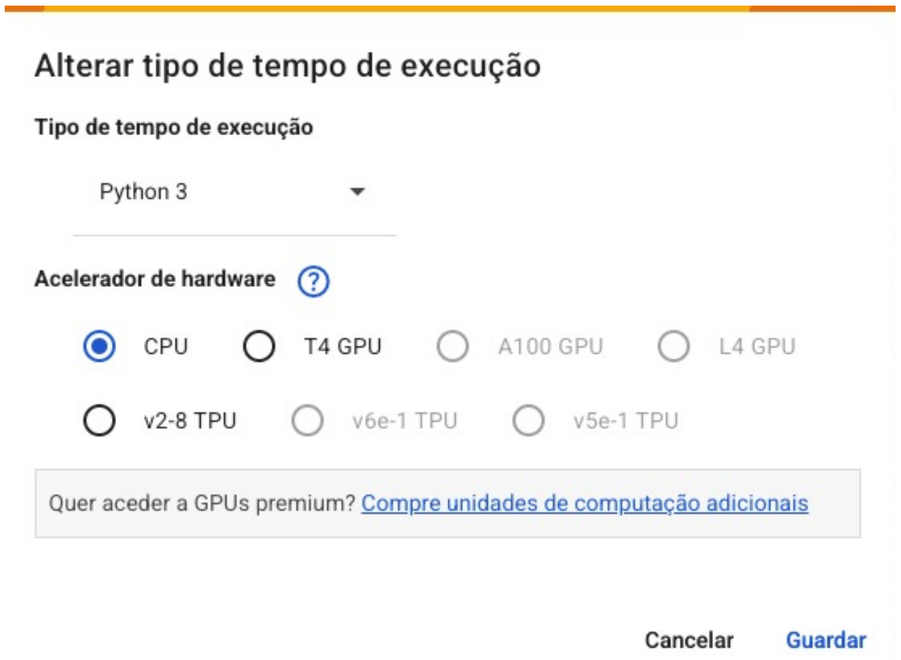


colab

TOOLING

Colab

Configurar tipo de runtime



TOOLING

Colab

- Aceder ao sistema de ficheiros local e ao Google Drive
  - https://colab.research.google.com/notebooks/io.ipynb
- Usar o Google Colab com o GitHub
  - https://colab.research.google.com/github/googlecolab/colabtools/blob/master/notebooks/colab-github-demo.ipynb
- Importar bibliotecas e Upgrading TensorFlow
  - https://colab.research.google.com/notebooks/snippets/importing_libraries.ipynb
- Usar GPUs e TPUs
  - https://colab.research.google.com/notebooks/gpu.ipynb
  - https://colab.research.google.com/notebooks/tpu.ipynb

colab

TOOLING

Colab

Quais são as limitações?

- Limites de recursos (RAM, horas de GPU, tempo limite de inatividade).
- As sessões são desconectadas após inatividade ou execuções longas.
- Requer conexão à Internet.
- Personalização limitada em comparação com ambientes locais.

Quando deve ser utilizado?

- Experiências rápidas em ML/DL.
- Aprender Python, ciência de dados e ML.
- Partilhar notebooks.
- Executar código em GPU sem *hardware* caro.

TOOLING

Kaggle Kernels

O que é o kaggle?

- Uma plataforma online gratuita para criar e partilhar ciência de dados.
- Permite a colaboração em kernels (notebooks).
- Famoso pelas competições de ciência de dados.
- Fornece conjuntos de dados, em diversos formatos, como CSV e JSON.
- Comunidade ativa de cientistas de dados que compartilham conhecimento, discutem projetos e oferecem suporte.

O que são kaggle kernels?

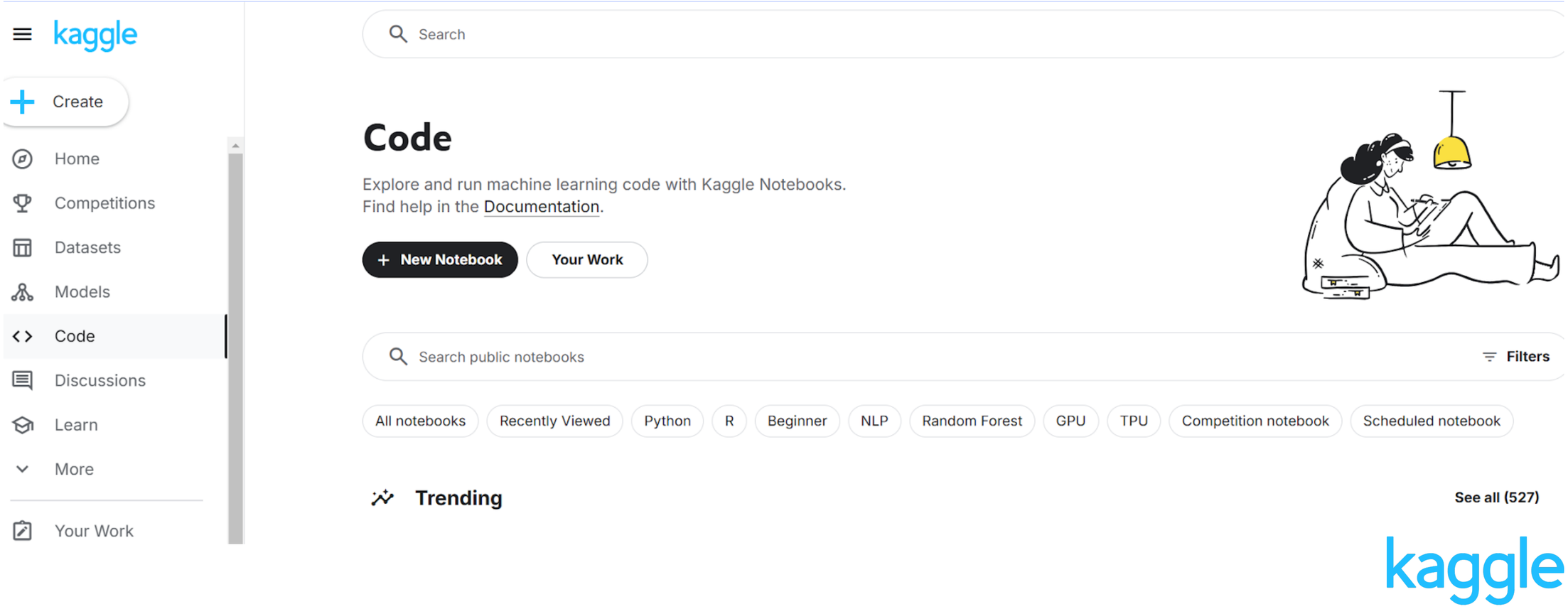
- Jupyter Notebooks baseados na *cloud* (sem necessidade de instalação).
- Suporte Python e R.
- Acesso gratuito a CPU, GPU e TPU para treinar modelos.
- Acesso direto a datasets do Kaggle.
- Partilha e colaboração fáceis.

kaggle

TOOLING

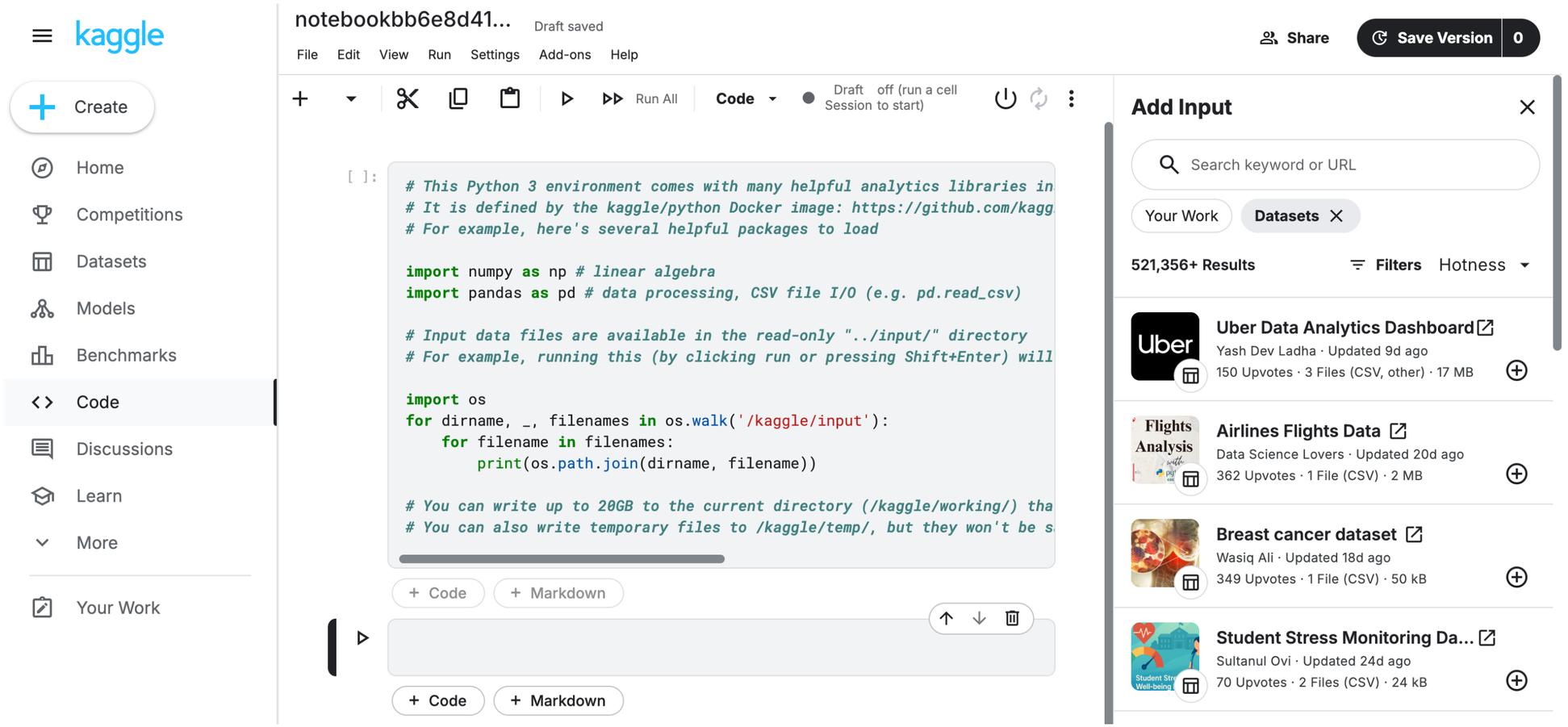
Kaggle Kernels

Collaborative Notebooks for Data Science & Machine Learning



TOOLING

Kaggle Kernels



kaggle

TOOLING

Kaggle Kernels

Quais são as limitações?

- Limites de recursos (por exemplo, sessões de GPU de 9 horas de tempo de execução, restrições de RAM/CPU - CPU de 4 núcleos com 17 GB de RAM e GPU de 2 núcleos com 14 GB de RAM).
- Menos flexível do que ambientes locais (bibliotecas limitadas).
- Requer conexão à Internet.

Quando deve ser utilizado?

- Prototipagem rápida de modelos de ML.
- Aprendizagem e experimentação com datasets.
- Participação em competições Kaggle.
- Partilha de notebooks com a comunidade.

kaggle

TOOLING

Overview

	ANACONDA	PyTorch	colab	kaggle
Instalação	Local	Biblioteca Python	Nenhuma (cloud)	Nenhuma (cloud)
Ambiente	Local	Local/Cloud	Cloud (Google)	Cloud (Kaggle)
Suporte a GPU	Sim (indireto)	Sim	Sim (grátis limitado)	Sim (grátis limitado)
Foco Principal	Gestão e isolamento de ambientes	*Framework* de ML	Prototipagem e ensino	Competição, datasets e colaboração
Colaboração	Baixa	Média	Alta	Alta

Python Revisão

Tipos de Dados

Integer

1
20
-7

Float

0.1
-0.5
6.43

String

“Hello World!”
”1”

Boolean

True
False

Python Revisão

Estruturas de Dados

Tuplo

coordenadas=(20, 80)

```
print(coordenadas[1]) #Output: 80
```

Um **tuplo** é uma coleção ordenada e imutável.

Lista

nomes = [“João”, “Maria”]

```
print(nomes[0]) #Output: João
```

Uma **lista** é uma coleção ordenada e mutável que permite elementos repetidos.

Set

numbers = {1, 2, 3, 2}

```
print(numbers) #Output: {1, 2, 3}
```

Um **set** é uma coleção não ordenada de elementos únicos.

Dicionário

aluno = {”nome”: “Inês”, ”idade": 22}

```
print(aluno["nome"])  #Output: Inês
```

Um **dicionário** é uma coleção não ordenada de pares chave-valor.

Python Revisão

Operadores aritméticos

Operador	Descrição	Exemplo (a = 10, b = 3)	Resultado
+	Soma	a + b	13
-	Subtração	a - b	7
*	Multiplicação	a * b	30
/	Divisão	a / b	3.33
//	Operador de divisão que fornece um inteiro como saída e descarta o resto	a // b	3
%	Modulo	a % b	1
**	Exponenciação	a ** b	1000

Python Revisão

Operadores de comparação

Operador	Descrição	Exemplo (a = 10, b = 3)	Resultado
==	Igual a	a == b	False
!=	Diferente de	a != b	True
>	Maior que	a > b	True
<	Menor que	a < b	False
>=	Maior ou igual que	a >= b	True
<=	Menor ou igual que	a <= b	False

Python Revisão

Outros operadores

Operadores de Atribuição

Operador	Descrição	Exemplo	Resultado
=	Atribuir	x = 5	-
+=	Adicionar e atribuir	x += 3	x = x + 3
-=	Subtrair e atribuir	x -= 3	x = x - 3
*=	Multiplicar e atribuir	x *= 3	x = x * 3
/=	Dividir e atribuir	x /= 3	x = x / 3

Operadores Lógicos

Operador	Descrição	Exemplo	Resultado
and	True se ambos forem true	True and False	False
or	True se pelo menos um for true	True or False	True
not	Inverte o resultado	not True	False

Python Revisão

Loops

If-elif-else Statement

```
x = 5
if x > 5:
    print("Greater")
elif x == 5:
    print("Equal")
else:
    print("Less")
```

Foor Loop

```
for i in range(3):
    print(i)
```

While Loop

```
count = 0
while count < 3:
    print(count)
    count += 1
```

Python Revisão

Loop Control Statements

break *sai do loop prematuramente*

```
for i in range(5):
    if i == 3:
        break
    print(i)
```

continue *ignora a iteração atual e continua com a seguinte.*

```
for i in range(5):
    if i == 2:
        continue
    print(i)
```

pass *não faz nada, é utilizado como placeholder.*

```
for i in range(5):
    if i == 2:
        pass
    print(i)
```

Python Revisão

Funções

Definir função

```
def greet(name):
    print(f"Hello, {name}!")
```

Função com parâmetro default

```
def greet(name="Student"):
    print(f"Hello, {name}!")
```

Executar função

```
greet()
greet("Alice")
```

Função com return

```
def add(a, b):
    return a + b

result = add(5, 3)
print(result)  # Output: 8
```

Python Revisão

Funções

Função com argumentos key-value

```
def student_info(name, age):
    print(f"Name: {name}, Age: {age}")

student_info(age=21, name="John")
```

Função com nº arbitrário de argumentos

args – argumentos não key-value

```
def total(*numbers):
    return sum(numbers)

print(total(1, 2, 3, 4))
```

****kwargs - argumentos key-value***

```
def print_info(**info):
    for key, value in info.items():
        print(f"{key}: {value}")

print_info(name="Alice", age=25)
```

Python Revisão

Exceptions

⚠ O bloco else é executado se não ocorrer nenhuma exceção. Enquanto o bloco finally é sempre executado.

Bloco try-except

```
try:
    x = int("abc")
except (ValueError, TypeError):
    print("Invalid input!")
```

Else com try-except

```
try:
    x = int("10")
except ValueError:
    print("Conversion failed.")
else:
    print("Conversion successful:", x)
```

Finally com try-except

```
try:
    f = open("test.txt", "r")
except FileNotFoundError:
    print("File not found.")
finally:
    print("Execution completed.")
```

Python Revisão

Exceptions

Exceções comuns do Python

ValueError: Valor inválido
TypeError: Tipo inválido
IndexError: Índice fora do intervalo
KeyError: Chave não encontrada no dicionário
FileNotFoundError: Ficheiro não encontrado

Usando a keyword raise

```
def divide(a, b):
    if b == 0:
        raise ValueError("Denominator cannot be zero")
    return a / b

print(divide(10, 2))
```

Definir uma exception personalizada

```
class AgeTooSmallError(Exception):
    pass

age = 12
if age < 18:
    raise AgeTooSmallError("You must be at least 18 years old.")
```

Python Revisão

Pandas Dataframe

`pd.read_csv('teste.csv')` – carrega um ficheiro csv para o dataframe

`df.head(n)` Seleciona as primeiras n linhas.

`df.tail(n)` Seleciona as primeiras n colunas.

`len(df)` - # de linhas no DataFrame

`df.shape` - Tuplo com o # de linhas, # de colunas do DataFrame.

`df['w'].nunique()` - # de valores distintos numa coluna.

`df['w'].unique()` - valores distintos numa coluna.

`df['w'].value_counts()` - contar o número de linhas com valor único para cada variável

Python Revisão

Pandas Dataframe

df.describe() – Estatísticas e descrições básicas para cada coluna.

df.info() – Imprime um resumo conciso do DataFrame.

df.infer_objects() – Tenta inferir um tipo melhor para dados do tipo objeto.

df.fillna(value) – Substitui todos os dados NA/nulos por esse valor.

df.drop(['c1','c2'], axis=1, inplace=True) – Remove as colunas do DataFrame

para remover colunas (se fosse axis=0, removeria linhas)

modifica o DataFrame diretamente em vez de devolver um novo.

Transformação de Dados

Dados Categóricos

- Dados categóricos são variáveis qualitativas que representam categorias ou grupos.
- Os dados categóricos são amplamente divididos em dois tipos:

Dados nominais
categorias sem ordem inerente

Exemplo
cores: vermelho, azul, verde

Dados ordinais
categorias com uma ordem natural

Exemplo
níveis de satisfação: baixo, médio, alto

- Estes dados precisam de ser convertidos em números antes de serem usados na maioria dos algoritmos de Machine Learning.

Transformação de Dados

One Hot Encoding

- Transforma variáveis **categóricas** em **binárias**.
- Cada categoria transforma-se numa nova **coluna** com valores 0 ou 1.
- Evita que o modelo interprete uma ordem ou hierarquia inexistente.

Cor
Verde
Azul
Amarelo
Verde

→

Verde	Azul	Amarelo
1	0	0
0	1	0
0	0	1
1	0	0

⚠ Pode levar ao aumento da dimensionalidade e, por isso, deve ser evitado em variáveis com alta cardinalidade.

⚠ Deve ser evitado quando há ordem natural nas categorias (ex.: Pequeno, Médio, Grande).

💡 Recomendado para variáveis categóricas **nominais** (ex: "Feminino", "Masculino", "Indeterminado").

Transformação de Dados

One Hot Encoding

Aplicação em Python (usando o Pandas):

```
One-Hot Encoded Data using Pandas:
   Cor_Amarelo  Cor_Azul  Cor_Verde
0        False     False       True
1        False      True      False
2         True     False      False
3        False     False       True
```

```python
import pandas as pd

# criar o DataFrame
df = pd.DataFrame({'Cor': ['Verde', 'Azul', 'Amarelo', 'Verde']})
# aplicar o OHE com o pandas
df_pd_ohe = pd.get_dummies(df, columns=['Cor'], drop_first=False)
print(f"One-Hot Encoded Data using Scikit-Learn:\n{df_pd_ohe}\n")
```

Em modelos lineares, manter todas as colunas pode causar **multicolinearidade** (*dummy variable trap*). O parâmetro `drop_first=True` no pandas.get_dummies() remove a **primeira coluna gerada**, evitando redundância entre as variáveis dummy.

```
One-Hot Encoded Data using Pandas:
   Cor_Azul  Cor_Verde
0     False       True
1      True      False
2     False      False
3     False       True
```

Transformação de Dados

One Hot Encoding

Aplicação em Python (usando o Scikit-Learn):

```
One-Hot Encoded Data using Scikit-Learn:
   Cor_Amarelo  Cor_Azul  Cor_Verde
0          0.0       0.0        1.0
1          0.0       1.0        0.0
2          1.0       0.0        0.0
3          0.0       0.0        1.0
```

```python
import pandas as pd
from sklearn.preprocessing import OneHotEncoder

# criar o DataFrame
df = pd.DataFrame({'Cor': ['Verde', 'Azul', 'Amarelo', 'Verde']})
# aplicar o OHE com o scikit-learn
encoder = OneHotEncoder(sparse_output=False)
encoded = encoder.fit_transform(df[['Cor']])
# gerar nomes de colunas apropriados, e.g.: ['Cor_Amarelo', 'Cor_Azul', 'Cor_Verde']
one_hot_df = pd.DataFrame(encoded, columns=encoder.get_feature_names_out(['Cor']))
# remover a coluna categórica original Cor do DataFrame e concatenar as restantes colunas com as novas colunas codificadas em one-hot lado a lado
df_sklearn_ohe = pd.concat([df.drop(['Cor'], axis=1), one_hot_df], axis=1)
print(f"One-Hot Encoded Data using Scikit-Learn:\n{df_sklearn_ohe}\n")
```

Transformação de Dados

One Hot Encoding

Aplicação em Python (usando o Scikit-Learn):

```
encoder = OneHotEncoder(sparse_output=False)
```

↪ **DENSE OUTPUT (sparse_output=False)**

- Retorna um **array NumPy** completo com todos os valores (0 e 1).
- Fácil de imprimir e inspecionar.
- Representação:

```
[[0. 0. 1.]
 [0. 1. 0.]
 [1. 0. 0.]
 [0. 0. 1.]]
```

↪ **SPARSE OUTPUT (sparse_output=True, valor por defeito)**

- Retorna uma **matriz esparsa** (do SciPy), que só guarda as posições onde há **1s**, ignorando todos os zeros.
- Muito mais **eficiente em termos de memória** quando existem muitas categorias.
- Representação:

```
Coords      Values
(0, 2)      1.0
(1, 1)      1.0
(2, 0)      1.0
(3, 2)      1.0
```

Transformação de Dados

Label Encoding

- Converte categorias em números inteiros.
- Cada categoria recebe um número único.
- Útil para algoritmos que aceitam variáveis ordinais ou numéricas.

Cor
Verde
Azul
Amarelo
Verde

→

Cor
2
1
0
2

⚠ O modelo pode interpretar que há uma ordem lógica entre os valores, i.e., o modelo pode interpretar erroneamente que Verde (2) > Azul (1) > Amarelo (0), mesmo não existindo hierarquia.

💡 Recomendado para variáveis categóricas **ordinais** (ex: "Baixo", "Médio", "Alto").

Transformação de Dados

Label Encoding

Aplicação em Python (usando o Scikit-Learn):

```
Label Encoded Data using Scikit-Learn:
       Cor  Cor_Cod
0    Verde        2
1     Azul        1
2  Amarelo        0
3    Verde        2
```

```python
from sklearn.preprocessing import LabelEncoder
import pandas as pd

# criar o DataFrame
df = pd.DataFrame({'Cor': ['Verde', 'Azul', 'Amarelo', 'Verde']})
# inicializar e aplicar o LabelEncoder
le = LabelEncoder()
df['Cor_Cod'] = le.fit_transform(df['Cor'])
print(f"Label Encoded Data using Scikit-Learn:\n{df}\n")
```

Transformação de Dados

Label Encoding

Aplicação em Python (usando o Pandas):

```
Label Encoded Data using Pandas:
       Cor  Cor_Cod
0    Verde        2
1     Azul        1
2  Amarelo        0
3    Verde        2
```

```python
import pandas as pd

# criar o DataFrame
df = pd.DataFrame({'Cor': ['Verde', 'Azul', 'Amarelo', 'Verde']})
# inicializar e aplicar o LabelEncoder
df['Cor_Cod'] = df['Cor'].astype('category').cat.codes
print(f"Label Encoded Data using Pandas:\n{df}\n")
```

Aplicação Prática

Ficha de exercícios 1



Blackboard